Autovalutazione — i cicli

Lezione coperta: `L06` — cicli: `while`

Quando farla. Da solo, **prima** della sessione `A02`. In aula la commentiamo insieme: chi arriva avendola fatta sa già su che cosa chiedere.

Come funziona. Due parti.

- **Otto domande a risposta multipla.** Sotto ciascuna trovi la risposta giusta **e il motivo per cui le altre sono sbagliate**: quelle spiegazioni sono la parte che insegna, quindi leggile anche quando hai indovinato.
- **Cinque programmi da scrivere.** Aprili nell'editor, uno per file, e **mandali in esecuzione**: è scrivendo il codice che scopri se hai capito davvero. Ogni programma ha un **caso piccolo**, che puoi controllare a mano mentre lo scrivi, e un **controllo**: un risultato che a mano non si ottiene, e che il tuo programma deve stampare uguale. Se lo stampa, il programma funziona; se no, rileggilo con il caso piccolo. La soluzione è sotto «Mostra la soluzione»: aprila **dopo**.

Non è un voto. Serve a scoprire che cosa non sai prima che te lo chieda qualcun altro. Se sbagli, segna il numero e portalo in aula.

Nessun programma usa `input()`: i dati sono scritti nelle prime righe, e per provare un altro caso basta cambiarli.

Parte 1 — Domande

1 — `if` o `while`

Che cosa stampa?

```
n = 3
if n > 0:
    n = n - 1
print(n)
```

Mostra la risposta

2: l'`if` controlla la condizione **una volta sola**.

- *`0`* — sarebbe la risposta con `while n > 0:`, che ripete finché la condizione resta vera. L'`if` esegue il blocco al massimo una volta e prosegue;
- *`3`* — no: la condizione `3 > 0` è vera, quindi il blocco viene eseguito e `n` diminuisce;
- *errore* — il codice è corretto.

È la differenza fra selezione e iterazione: le due righe si scrivono quasi uguali, e fanno cose diverse.

---

2 — Il contatore dopo il ciclo

Che cosa stampa?

```
i = 0
while i < 5:
    i = i + 1
print(i)
```

Mostra la risposta

`5`: il ciclo si ferma proprio quando `i` vale 5, perché `5 < 5` è falsa.

- *`4`* — è l'ultimo valore per cui la condizione era vera, ma dentro quel giro `i` è stato incrementato ancora;
- *`6`* — no: quando `i` arriva a 5 il ciclo non entra più, e nessuno lo incrementa.

Dopo un `while`, la variabile della condizione vale **il primo valore che rende falsa la condizione**.

---

3 — Dentro o fuori

Quante righe stampa?

```
i = 1
somma = 0
while i <= 4:
    somma = somma + i
    i = i + 1
    print(somma)
```

Mostra la risposta

Quattro: `1`, `3`, `6`, `10`.

- *una, `10`* — sarebbe così con il `print` **fuori** dal ciclo, cioè senza i quattro spazi davanti. Qui è dentro, e si esegue a ogni giro;
- *cinque* — i giri sono quattro, con `i` che vale 1, 2, 3, 4; quando vale 5 il ciclo non entra.

L'indentazione decide che cosa si ripete.

---

4 — Quale non finisce

Quale di questi quattro cicli **non finisce mai**?

A

```
i = 0
while i < 3:
    print(i)
    i = i + 1
```

B

```
i = 0
while i < 3:
    print(i)
i = i + 1
```

C

```
i = 10
while i > 0:
    i = i - 3
```

D

```
i = 5
while i < 3:
    i = i + 1
```

Mostra la risposta

B: l'incremento non ha i quattro spazi davanti, quindi **non fa parte del ciclo**. Dentro, `i` resta 0 per sempre, e il programma stampa `0` all'infinito. Si interrompe con `ctrl + c`.

- *A* — è il ciclo corretto: stampa 0, 1, 2 e si ferma;
- *C* — `i` vale 10, 7, 4, 1, poi `-2`: a quel punto `i > 0` è falsa e il ciclo finisce. Non passa per lo zero, ma la condizione è `>` e non `!=`;
- *D* — finisce subito: `5 < 3` è falsa già al primo controllo, e il ciclo fa **zero giri**.

---

5 — Un quarto alla volta

In aula abbiamo visto che sommando `0.1` dieci volte non si arriva esattamente a `1`. Questo ciclo finisce?

```
x = 0
giri = 0
while x != 1:
    x = x + 0.25
    giri = giri + 1
print(giri)
```

Mostra la risposta

Sì, dopo 4 giri: `0.25` in binario si scrive **esatto**, perché è un quarto, cioè una potenza di 2. Le somme sono esatte e `x` vale proprio `1`.

- *no, come con `0.1`* — `0.1` in binario ha infinite cifre, come `1/3` in decimale, e ogni somma porta un piccolo errore. `0.25` no;
- *dipende dal computer* — no: la rappresentazione dei float è la stessa su tutti i calcolatori.

Funziona, ma è fortuna: basta cambiare il passo e il ciclo non finisce più. Per questo un ciclo si conta con gli interi, e i float non si confrontano con `==` o `!=`.

---

6 — Uscire senza `break`

Il programma cerca il più piccolo divisore di `91` maggiore di `1`. Che cosa stampa?

```
n = 91
d = 2
trovato = False
while d < n and not trovato:
    if n % d == 0:
        trovato = True
    d = d + 1
print(d)
```

Mostra la risposta

`8`, anche se il divisore è `7`.

- *`7`* — quando `d` vale 7 il resto è zero e `trovato` diventa `True`, ma la riga dopo l'`if` **si esegue comunque**: `d` passa a 8. Il ciclo si ferma al controllo successivo;
- *`91`* — no: il ciclo si ferma prima, appena `trovato` è vero.

Uscire con la condizione, senza `break`, vuol dire che **il giro in corso finisce**. Stampare `d - 1` non basta: con `n = 97`, che è primo, nessun resto è zero, il ciclo esce per **l'altra** condizione, `d < n`, e `d - 1` vale `96`. La correzione è far avanzare `d` solo finché non ha trovato, con `d = d + 1` in un `else`: allora stampa `7` con `91` e `97` con `97`. Quando un `while` ha due condizioni, dopo il ciclo chiediti **quale delle due lo ha fermato**.

---

7 — La sentinella sommata

Il programma dovrebbe fare la media dei numeri inseriti, fermandosi al primo negativo. Si scrive `4`, `6`, `-1`. Che cosa stampa?

```
somma = 0
n = 0
valore = 0
while valore >= 0:
    valore = int(input("numero (negativo per finire): "))
    somma = somma + valore
```

```
    n = n + 1
print(somma / n)
```

Mostra la risposta

`3.0`: la media giusta sarebbe `5.0`.

- *`5.0`* — sarebbe giusto se il `-1` non entrasse nel conto. Qui invece il valore viene **letto, sommato e contato prima** che la condizione lo controlli: `somma` vale `4 + 6 - 1 = 9`, `n` vale 3;
- *un errore* — nessun errore: il programma risponde, ed è sbagliato.

La sentinella va controllata **prima** di usarla: si legge il primo valore prima del ciclo, e i successivi in fondo al corpo.

---

8 — Una variabile che non c'è

Che cosa succede?

```
i = 0
while i < 0:
    ultimo = i
    i = i + 1
print(ultimo)
```

Mostra la risposta

Un errore, `NameError: name 'ultimo' is not defined`.

- *stampa `0`* — `ultimo` verrebbe assegnato solo **dentro** il ciclo, e il ciclo non entra mai: `0 < 0` è falsa. La riga `ultimo = i` non viene eseguita;
- *stampa `None`* — Python non inventa un valore per una variabile mai assegnata: si ferma.

Una variabile assegnata solo dentro un ciclo, o solo in un ramo di un `if`, dopo potrebbe non esistere. Chiediti sempre: **e se il ciclo non è mai entrato?**

Parte 2 — Programmi

Ogni programma va scritto in un file suo, per esempio `a02_p1.py`, ed eseguito con `python3 a02_p1.py`. Usa solo quello che hai visto: `while`, `if`, gli operatori. Niente `break`: un ciclo finisce quando la sua condizione diventa falsa.

Programma 1 — I pari fra due numeri

Dati due interi `a` e `b`, **in qualunque ordine**, stampa quanti sono i numeri pari compresi fra i due, **estremi inclusi**, e la loro somma.

```
a = 7
b = 2
```

Caso piccolo: con `a = 7` e `b = 2` i pari sono 2, 4, 6: il programma stampa **3** pari, somma **12**. Scambiando `a` e `b` deve stampare lo stesso.

Controllo: con `a = 9871` e `b = 123` deve stampare **4874** pari, somma **24355378**.

Mostra la soluzione

```
a = 9871
b = 123

# il ciclo va dal piu' piccolo al piu' grande: se sono al contrario, si scambiano
if a > b:
    t = a
    a = b
    b = t

# il primo pari: a stesso se e' pari, altrimenti quello dopo
x = a
if x % 2 != 0:
    x = x + 1

quanti = 0
somma = 0
while x <= b:
    quanti = quanti + 1
    somma = somma + x
    x = x + 2
print("pari:", quanti, "somma:", somma)
```

Tre punti dove si sbaglia:

- **l'ordine**: senza lo scambio, con `a = 9871` e `b = 123` la condizione `x <= b` è falsa subito, il ciclo fa zero giri e il programma stampa `0` pari;
- **gli estremi**: con `x < b` invece di `x <= b`, se `b` è pari resta fuori. Nel caso piccolo `b` vale 7 e non si vede: prova `a = 2`, `b = 8`, che deve dare 4 pari, somma 20;
- **il passo**: partire dal primo pari e avanzare di 2 fa metà dei giri. Funziona anche avanzare di 1 con un `if x % 2 == 0` dentro il ciclo: è più lento, ma è giusto.

Programma 2 — Il capitale che cresce

Un capitale di 1000 € è investito al **3,7% annuo**. Ogni anno il capitale diventa

```
capitale = capitale * (1 + tasso / 100)
```

Dopo quanti anni il capitale **supera** 2500 €? E quanto vale in quel momento?

```
capitale = 1000
tasso = 3.7
obiettivo = 2500
```

Caso piccolo: con `tasso = 100` il capitale raddoppia ogni anno, 2000 € dopo un anno e 4000 € dopo due: il programma stampa **2** anni, **4000.0** €: con il tasso il capitale diventa un float.

Controllo: con i dati sopra deve stampare **26** anni, **2571.86** € (arrotondato ai centesimi con `round(capitale, 2)`).

Mostra la soluzione

```
capitale = 1000
tasso = 3.7
obiettivo = 2500

anni = 0
while capitale <= obiettivo:
    capitale = capitale * (1 + tasso / 100)
    anni = anni + 1
print("anni:", anni, "capitale:", round(capitale, 2))
```

Due punti dove si sbaglia:

- **supera**, cioè `>`: il ciclo deve continuare finché il capitale è **minore o uguale**, quindi la condizione è `capitale <= obiettivo`. Con `capitale < obiettivo` si fermerebbe anche a 2500 esatti, che non è superare;
- **il contatore**: `anni` parte da 0 e cresce nello stesso giro del capitale. Se parte da 1, la risposta è 27.

Il numero di giri non si conosce in anticipo: è il caso tipico del `while`.

Programma 3 — Il volo di un numero

Si parte da un intero positivo `n` e si ripete questa regola finché `n` non diventa 1:

```
if n % 2 == 0:
    n = n // 2
else:
    n = 3 * n + 1
```

cioè: se è pari si dimezza, se è dispari si moltiplica per 3 e si aggiunge 1. Stampa **quanti passi** servono per arrivare a 1 e **il valore più alto** toccato lungo il percorso, contando anche quello di partenza.

```
n = 6
```

Caso piccolo: da 6 il percorso è 6, 3, 10, 5, 16, 8, 4, 2, 1: **8** passi, massimo **16**.

Controllo: da `n = 27` deve stampare **111** passi, massimo **9232**.

Mostra la soluzione

```
n = 27

passi = 0
massimo = n
while n != 1:
    if n % 2 == 0:
        n = n // 2
    else:
        n = 3 * n + 1
    passi = passi + 1
    if n > massimo:
        massimo = n
print("passi:", passi, "massimo:", massimo)
```

Tre punti dove si sbaglia:

- **il massimo parte dal primo valore**, `massimo = n`, non da 0: con 0 il valore di partenza non viene mai confrontato, perché il confronto sta dopo il passo. Con `n = 27` non si vede, perché il massimo arriva dopo; con `n = 16` il percorso è 16, 8, 4, 2, 1 e il programma stampa massimo `8`;
- `//` **e non** `/`: con `/` i passi tornano lo stesso, perché si divide solo un numero pari, ma `n` diventa un float e il massimo esce `9232.0`. Per dividere un intero e restare negli interi si usa `//`;
- **contare i passi, non i numeri**: il percorso da 6 tocca 9 numeri ma fa 8 passi. Se il contatore parte da 1, la risposta è 112.

Nessuno sa dimostrare che questo ciclo finisca per ogni `n`: è un problema aperto della matematica, la **congettura di Collatz**. Per tutti i numeri provati finora, finisce.

Programma 4 — Il massimo comun divisore per sottrazioni

Il massimo comun divisore di due interi positivi `a` e `b` si trova così: **finché sono diversi**, si toglie il più piccolo dal più grande. Quando diventano uguali, quel valore è il MCD. Stampa il MCD e **quante sottrazioni** sono servite.

```
a = 12
b = 8
```

Caso piccolo: 12 e 8 diventano 4 e 8, poi 4 e 4: MCD **4**, in **2** sottrazioni.

Controllo: con `a = 123456` e `b = 7890` deve stampare MCD **6**, **62** sottrazioni.

Mostra la soluzione

```
a = 123456
b = 7890

sottrazioni = 0
while a != b:
    if a > b:
        a = a - b
    else:
        b = b - a
    sottrazioni = sottrazioni + 1
print("MCD:", a, "sottrazioni:", sottrazioni)
```

È lo stesso problema dell'esercizio 11 di `L06_esercizi` («Divisori comuni») risolto con **un altro algoritmo**: lì si provano tutti i divisori, qui non se ne prova nessuno. Il numero di giri non si vede a occhio, e il ciclo finisce perché a ogni giro uno dei due numeri diminuisce e nessuno dei due scende sotto il MCD.

Funziona solo con interi **positivi**: con uno zero, `a - 0` resta `a` e il ciclo non finisce più.

Programma 5 — Le coppie

Quante coppie di interi `i` e `j`, con **`i` minore di `j`** e tutti e due fra 1 e `n`, hanno una somma **multipla di 7**?

```
n = 7
```

Caso piccolo: con `n = 7` le coppie sono (1, 6), (2, 5), (3, 4): **3**.

Controllo: con `n = 100` deve stampare **707**.

Mostra la soluzione

```
n = 100

coppie = 0
i = 1
while i <= n:
    j = i + 1
    while j <= n:
        if (i + j) % 7 == 0:
            coppie = coppie + 1
        j = j + 1
    i = i + 1
print("coppie:", coppie)
```

Tre punti dove si sbaglia:

- **`j = i + 1` dentro il ciclo esterno**: fuori, `j` non riparte, e il ciclo interno gira solo per il primo valore di `i`. Con `n = 100` il programma stampa 14. È il contatore che non riparte, visto in aula con le tabelline;
- **`j` parte da `i + 1`, non da 1**: da 1 ogni coppia si conta due volte, (2, 5) e (5, 2), e anche le coppie con `i` uguale a `j`. Con `n = 7` verrebbero 7 invece di 3;
- **le parentesi** in `(i + j) % 7`: senza, `%` si calcola prima di `+`, e si controlla `i + (j % 7)`.

Che cosa portare in aula

Segna i numeri che hai sbagliato, e i programmi che non danno il controllo. Se sono raggruppati, dicono qualcosa:

se hai sbagliato	rivedi
1, 2, 3, 4	come funziona un `while`: quando controlla, che cosa ripete, quando finisce — `L06_a`, *while* e *Trace table*
5, 6	come si esce da un ciclo, e perché i float non si contano — gli appunti di `F05` sui float
7, 8, programma 1	i bordi: zero giri, gli estremi, la sentinella — `L06_a`, *Sentinella*; `L06_esercizi` 5–8
programmi 2, 3, 4	contatori, accumulatori, il massimo — `L06_a`, *Contatori* e *Accumulatori*
programma 5	i cicli annidati — `L06_a`, *Annidati*